Task 1 – Analysis of Nuclear Plants Dataset

1.1 Loading the Data and Missing Values

```
[12] df.show()

+------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+
|Status|Power_range_sensor_1|Power_range_sensor_2|Power_range_sensor_3 |Power_range_sensor_4|Pressure _sensor_1|Pressure _sensor_2|Pressure _sensor_3|Pressure _sensor_4|Vibration_sensor_1|Vibration_sensor_2|Vibration_sensor_3|Vibration_sensor_4|
+------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+
|Normal|                4.5044|                0.7443|                 6.34|                1.9052|               29.5315|                0.8047|                2.2044|                 6.048|               14.4659|                21.648|                15.3429|                 1.2186|
|Normal|                4.4284|                0.9073|                5.6433|                1.6232|               27.5032|                1.4704|                1.9929|                5.9856|               20.8356|                0.0046|                14.8813|                 7.3483|
|Normal|                4.5291|                1.0190|                 6.113|                1.8565|               26.4271|                1.6247|                 1.942|                6.7162|                5.3358|                11.8779|                25.8014|                 0.2488|
|Normal|                5.1727|                1.0007|                7.0509|                0.2765|               25.1576|                 2.609|                2.9234|                 6.7485|                1.9017|                 1.8463|                 28.664|                 4.0157|
|Normal|                5.2258|                0.6125|                7.9504|                0.1547|               24.0765|                3.2113|                4.4563|                 5.8411|                0.5077|                  9.37|                34.8122|                13.4986|
|Normal|                4.8834|                0.5786|                5.6021|                0.5042|               21.0365|                2.6275|                4.3996|                 5.3487|                3.6465|                9.3793|                36.2431|                 11.124|
|Normal|                5.7422|                0.8862|                5.1013|                0.9402|               15.5171|                  2.92|                2.7061|                 5.0366|                9.1854|                9.1116|                27.3162|                 2.8526|
|Normal|                6.5076|                0.5438|                 7.153|                0.6711|               11.1481|                4.2967|                3.5731|                 4.6123|               14.7705|                8.0038|                24.9714|                 2.1417|
|Normal|                 5.625|                0.8403|                7.7867|                0.5214|                8.7975|                2.5395|                 4.788|                 5.365|                4.3588|                2.4187|                 20.9187|                 3.1253|
|Normal|                4.0042|                0.4002|                6.3604|                0.6031|                 6.189|                0.1537|                3.4034|                5.6573|                6.8117|               13.3581|                 23.5836|                 0.7478|
|Normal|                5.9079|                0.9229|                5.3613|                0.3516|                4.0913|                1.1409|                1.4172|                4.4551|                8.1422|                9.8039|                 20.798|                 0.1108|
|Normal|                6.4215|                0.5978|                5.5765|                1.1847|                1.3588|                2.2807|                2.8529|                 4.5802|                 0.007|                8.7331|                27.2793|                  3.114|
|Normal|                5.0253|                0.2161|                6.3439|                0.8835|                 4.0160|                1.3687|                3.0973|                 5.3891|               18.3585|               14.4843|                25.4514|                 5.0267|
|Normal|                5.5783|                0.6202|                6.3566|                0.8747|                 9.9027|                1.3455|                5.7394|                 4.1904|               16.6814|               16.8061|                23.3743|                 2.6033|
|Normal|                 5.495|                0.4071|                5.2286|                1.1903|               14.9418|                1.6888|                6.5821|                 2.7535|                 11.0|                16.3399|                  7.136|                 6.4808|
|Normal|                5.6337|                0.5182|                4.3826|                1.8375|               10.1719|                2.3323|                 6.841|                 3.5384|               13.7643|                20.014|                  4.8804|                20.5403|
|Normal|                5.8438|                1.1902|                5.1022|                2.0297|               20.4563|                3.9257|                9.0478|                 3.8586|               12.3057|               17.4384|                29.7256|                21.8234|
|Normal|                5.4053|                1.8027|                5.9399|                1.5262|               18.5719|                 3.660|               11.0384|                 3.5604|               10.6625|               10.4963|                24.8606|                16.2476|
|Normal|                5.2075|                1.4804|                5.8466|                1.1105|               16.7177|                2.6853|                8.0135|                 4.2600|               20.7064|                 7.893|                 7.8886|                24.1408|
|Normal|                6.0489|                0.9576|                5.3467|                1.5135|               15.4675|                3.7974|                7.7415|                 4.0424|               23.3928|                3.3972|                  17.54|                27.5656|
+------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+-----------------------+
only showing top 20 rows
```

Figure 1 Pyspark Dataframe of loaded csv file

Analysing a dataset with missing values can cause problems with the results provided such as wrong predictions of the later classifiers, therefore it is beneficial to check for and remove anomalies in the dataset, so they do not affect the final analysis results. Some examples of missing values from the dataset could be the data initially not being recorded or the data could have been lost since, therefore it is vital to check a dataset for missing values.

Having loaded the csv spreadsheet file named 'nuclear_plants_small_dataset' into a Pyspark dataframe as shown in the figure above, all the data appears to be the same as those records in the dataset (only the top 20 records/rows are presented with the '*df.show()*' command.

```
df2 = df.groupBy("Status").count()
df2.show()

+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  498|
|  Normal|  498|
+--------+-----+
```

Figure 2 groupBy method for number of records per category

Since not knowing how many categories exist in the dataset, I used the '*groupBy()*' method to create a secondary DataFrame showing the number of records per category under the 'Status' field. As shown in figure 2, there seems to be an equal distribution between abnormal and normal entries in the dataset. Likewise, there is also no mention of a '*null*' category, therefore there are no empty/missing values in the 'Status' field.

```
[15] df2 = df.groupBy("Status").count()
     df2.show()

+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  498|
|  Normal|  498|
+--------+-----+

df3 = df.dropna(how = "any")
df4 = df3.groupBy("Status").count()
df4.show()

+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  498|
|  Normal|  498|
+--------+-----+
```

Figure 3 using Pyspark's dropna() method for DataFrames

To further check if there are any missing values in the data, I used Pyspark's '*dropna()*' method for DataFrames as shown in figure 3. This method removes all the rows from the dataset that contain a null value for any variable, and therefore since figure 3 shows the before and after results of the '*dropna()*' method and the result showing no change in the number of records for both abnormal and normal Status labels', there appears to be no rows in the dataset that contain any null values.

To deal with missing values in the data, as shown above, a method of approaching this could be to remove all the rows in the dataset that include a null value. However, as concluded through Carlos Escobar's research, this may not be a sufficient approach since "*while deleting rows/columns or imputing missing values are common approaches, they may have serious implications such as loosing most of the training data (as demonstrated in the real case studies) or biasing the* estimates" (Escobar et al., 2020). Or in an example where the ".show()"

command presents numerous similar outputs for the datasets' fields – such as "abnormal" and "Abnormal" could be considered the same, but due to the difference in capitalisation, they would be considered different Status's. Overcoming this issue would be through utilising Pyspark DataFrames' replace method, where a dictionary serving as a map would be used as a guide for the replacement fields in the dataset column. For example, a dictionary of {"abnormal": "Abnormal"} would indicate the rows that have "abnormal" as their Status column, to be replaced with the capitalised version of "Abnormal".

I originally created these lines of code under a google collab browser since it displayed the contents of "*df.show()*" neater on the browser, however as shown later in this document, I transferred to a Jupyter notebook for the further tasks since the boxplots appeared neater than on the google collab.

1.2 Summary Statistics

1.2.1 Summary Statistics for the Normal Group

	Power_range_sensor_1	Power_range_sensor_2	Power_range_sensor_3	Power_range_sensor_4	Pressure _sensor_1	Pressure _sensor_2	Pressure _sensor_3	Pressure _sensor_4
min	0.085100	0.040300	4.382600	0.154700	0.024800	0.010400	0.077400	0.00580
max	12.129800	11.928400	14.098200	16.356800	56.856200	9.221200	12.647500	15.10850
mean	5.602453	6.844503	9.292054	8.701398	13.797526	3.415646	5.923353	5.58618
50%	5.178650	6.717650	9.262850	9.240850	10.634800	3.113000	5.739400	4.25915

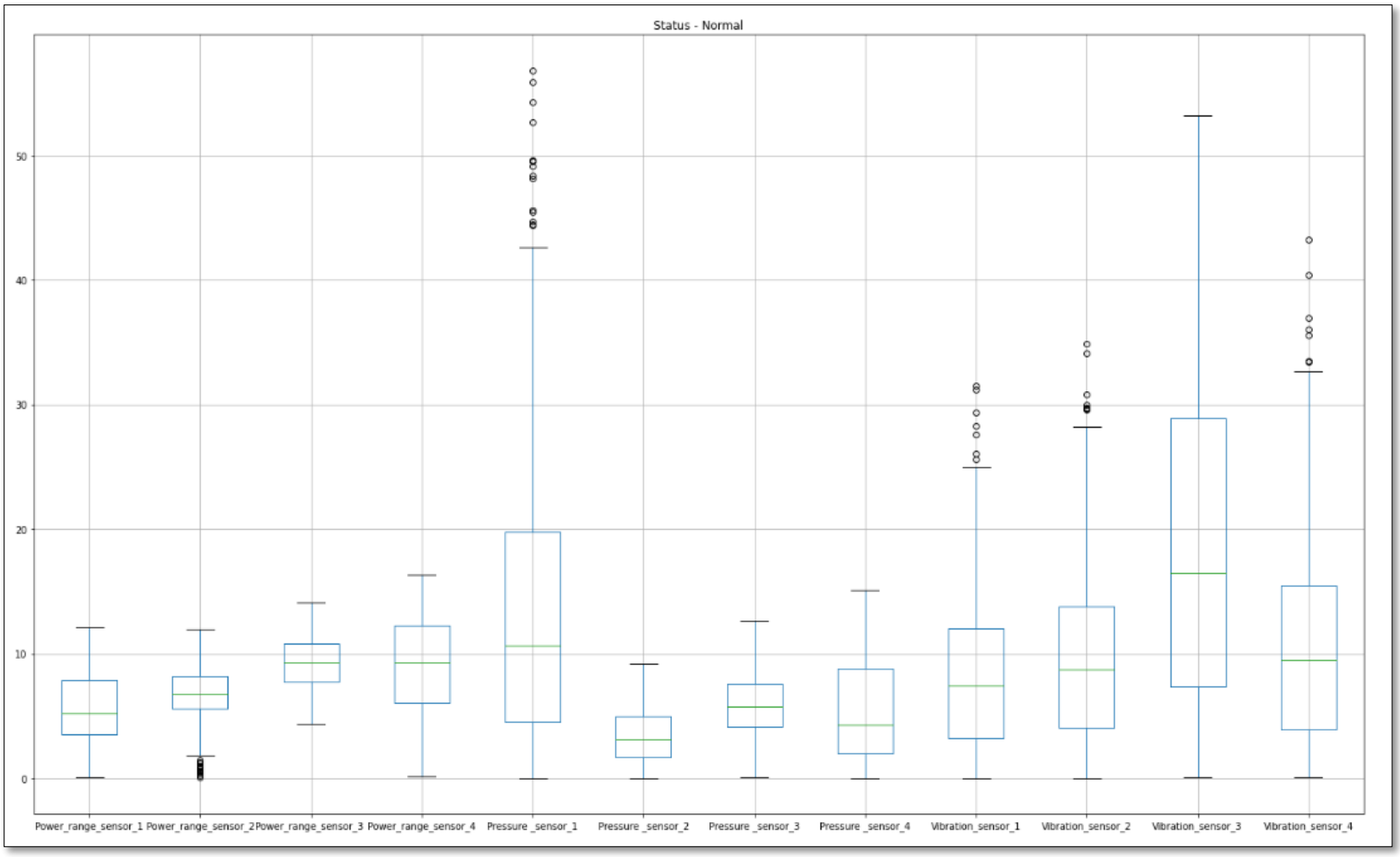
Vibration_sensor_1	Vibration_sensor_2	Vibration_sensor_3	Vibration_sensor_4
0.009200	0.027700	0.064600	0.083100
31.498100	34.867600	53.238400	43.231400
8.441437	9.699616	19.437804	10.925098
7.449900	8.700750	16.464500	9.485450

Analysing the dataset's contents could provide an insight to the variation of the data, for in the case of this dataset – for scientists or data analysts to interpret the various measurements of the reactor parts.

The above results were generated using the pandas library since it produced a more readable output in a more organised table compared to the results given through the data frame table in my first attempt in figure 4. This alternative output did not make it clear that the results provided were organised so that each row represented each column in the database file – for example, the first row of; [minimum: 0.0851, maximum: 12.1298] represented the calculated statistics for the first database column 'Power_range_sensor_1" – excluding the databases' first 'State' column of datatypes "Normal"/"Abnormal".

Minimum	Maximum	Mean	Median
0.0851	12.1298	5.602452811244976	5.1727
0.0403	11.9284	6.844503413654616	6.6998
4.3826	14.0982	9.292054016064245	9.2624
0.1547	16.3568	8.701398192771098	9.2404
0.0248	56.8562	13.797525502008051	10.6274
0.0104	9.2212	3.4156463855421695	3.113
0.0774	12.6475	5.923352610441759	5.7394
0.0058	15.1085	5.586180120481923	4.2574
0.0092	31.4981	8.441436947791166	7.4222
0.0277	34.8676	9.699615863453822	8.6684
0.0646	53.2384	19.437804417670666	16.4414
0.0831	43.2314	10.925097590361439	9.4347

Figure 4 Data Frame first attempt output

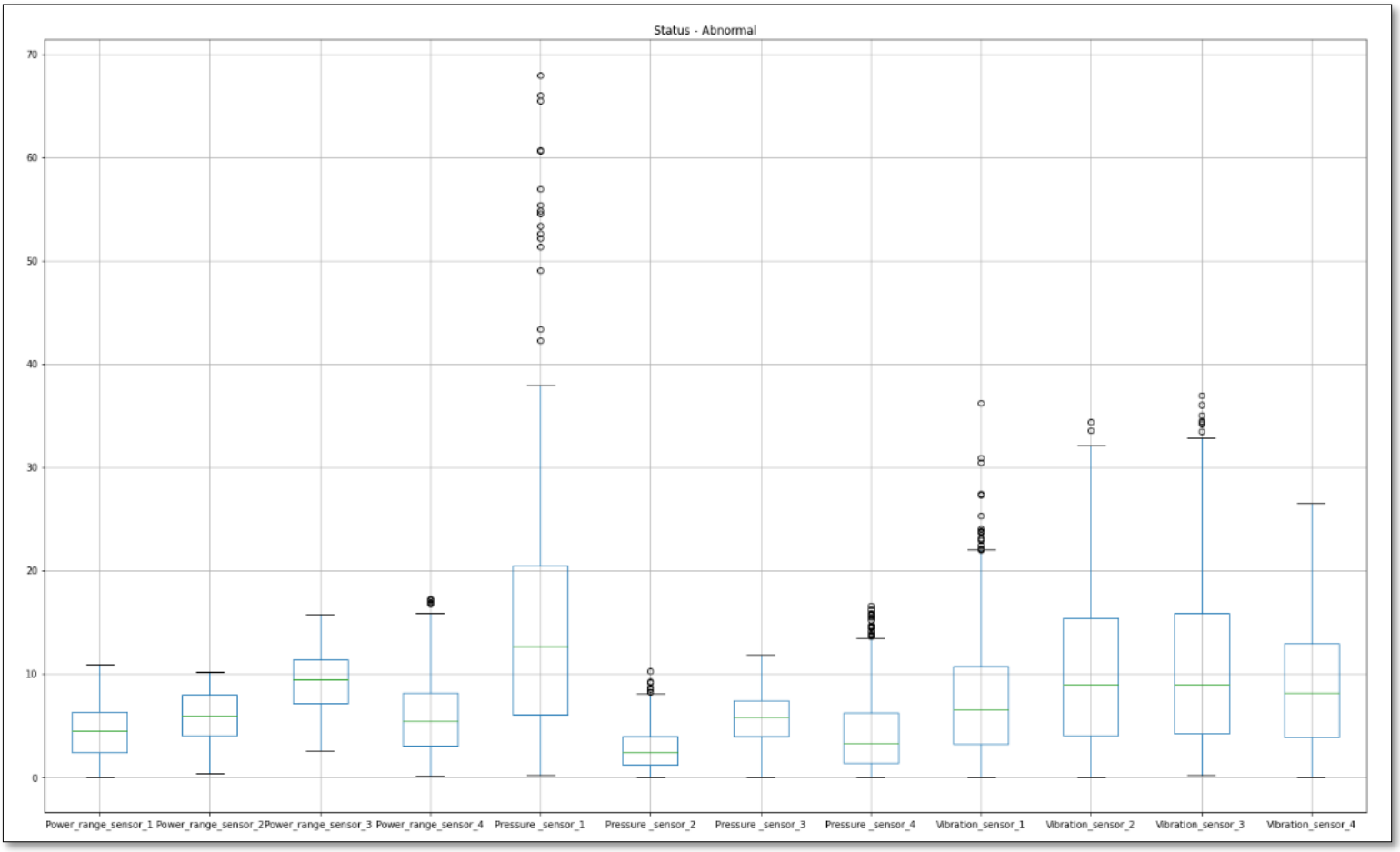


The boxplot above represents the distributions of data for each of the database's columns – as referenced along the x-axis, by representing each of the data's median with the middle green line, the lower and upper quartiles represented by the bottom and top blue outer lines/edges, the minimum of the dataset at the lower whisker and the maximum of the dataset at the higher whisker. All the remaining circles in alignment with the boxplot that fall beyond the minimum or maximum whisker represent outliers of the dataset.

1.2.2 Summary Statistics for the Abnormal Group

	Power_range_sensor_1	Power_range_sensor_2	Power_range_sensor_3	Power_range_sensor_4	Pressure _sensor_1	Pressure _sensor_2	Pressure _sensor_3	Pressure _sensor_4
min	0.008200	0.389100	2.583966	0.062300	0.131478	0.008262	0.001224	0.029478
max	10.923078	10.154100	15.759900	17.235858	67.979400	10.242738	11.772400	16.555620
mean	4.396695	5.914043	9.164170	6.009146	14.600728	2.740270	5.575115	4.407824
50%	4.513550	5.932218	9.472050	5.399300	12.596150	2.382689	5.744257	3.322575

Vibration_sensor_1	Vibration_sensor_2	Vibration_sensor_3	Vibration_sensor_4
0.000000	0.018500	0.131784	0.009200
36.186438	34.331466	36.911454	26.466900
7.887689	10.303570	10.938159	8.942085
6.535950	8.973100	8.987269	8.137600



1.3 The Relationship between Features

```
dataPanda = pd.read_csv("nuclear_plants_small_dataset.csv")
corr = dataPanda.corr(method='pearson')
# calculates a pairwise correlation of columns in the dataset, "pe
corr.style.background_gradient(cmap = 'coolwarm')
# setting the style of the correlation matrix to be displayed as a
```

	Power_range_sensor_1	Power_range_sensor_2	Power_range_sensor_3	Power_range_sensor_4	Pressure _sensor_1	Pressure _sensor_2	Pressure _sensor_3	Pressure _sensor_4	Vibration_sensor_1	Vibration_sensor_2	Vibration_sensor_3	Vibration_sensor_4
Power_range_sensor_1	1.000000	0.095235	0.229943	0.499795	0.047805	0.652081	0.199811	0.406382	-0.152247	-0.003684	0.106583	0.014177
Power_range_sensor_2	0.095235	1.000000	0.693818	0.228438	-0.222160	0.112438	0.583086	0.136380	-0.119142	0.011013	0.243242	0.058244
Power_range_sensor_3	0.229943	0.693818	1.000000	0.083362	-0.255865	0.166549	0.362688	0.062479	-0.249057	-0.038320	0.215811	-0.023962
Power_range_sensor_4	0.499795	0.228438	0.083362	1.000000	0.150635	0.567705	-0.070547	0.823637	-0.114117	0.043383	0.330499	-0.018193
Pressure _sensor_1	0.047805	-0.222160	-0.255865	0.150635	1.000000	0.116859	-0.118713	0.131136	0.101438	-0.035196	-0.084575	-0.125128
Pressure _sensor_2	0.652081	0.112438	0.166549	0.567705	0.116859	1.000000	0.091341	0.571403	-0.099950	-0.027698	0.011015	0.001189
Pressure _sensor_3	0.199811	0.583086	0.362688	-0.070547	-0.118713	0.091341	1.000000	0.001591	0.016497	0.029766	0.019993	0.105642
Pressure _sensor_4	0.406382	0.136380	0.062479	0.823637	0.131136	0.571403	0.001591	1.000000	-0.112696	0.010560	0.191842	0.047000
Vibration_sensor_1	-0.152247	-0.119142	-0.249057	-0.114117	0.101438	-0.099950	0.016497	-0.112696	1.000000	-0.045857	-0.076277	0.026480
Vibration_sensor_2	-0.003684	0.011013	-0.038320	0.043383	-0.035196	-0.027698	0.029766	0.010560	-0.045857	1.000000	0.103704	-0.023290
Vibration_sensor_3	0.106583	0.243242	0.215811	0.330499	-0.084575	0.011015	0.019993	0.191842	-0.076277	0.103704	1.000000	-0.015536
Vibration_sensor_4	0.014177	0.058244	-0.023962	-0.018193	-0.125128	0.001189	0.105642	0.047000	0.026480	-0.023290	-0.015536	1.000000

Due to the nature of the correlation matrix, all features where the column and row belong to the same feature result in a correlation of 1/red since they belong to the same feature. However, it appears that “Power_range_sensor_4” has the strongest correlation to “Pressure_sensor_4” with a pearson correlation of 0.8236, meaning there is a nearer to perfect positive linear correlation between the two features. As opposed to this, the closer the correlation float is to -1, this would represent the features are closer to a perfect negative linear correlation, such as “Pressure_sensor_1” with “Power_range_sensor_3” with its negative 0.0255 correlation. Within this correlation range [-1,1], the middle of the range being 0 would represent that there is no linear correlation between the features, and therefore “Pressure_sensor_2” and “Vibration_sensor_4” have a correlation of 0.001189, meaning it has one of the weakest relationships in this matrix between features (Zach, 2020).

1.4 Training Testing Split

Since this selection of code imports from the pyspark.ml.feature library, utilising its ".randomSplit" function, unless a seed is specified as it's parameter to indicate to always yield the same random splits "*since the partitioning of the PySpark DataFrame is the same*" (Inada, 2022).

```
from pyspark.ml.feature import VectorAssembler, StringIndexer
assem = VectorAssembler(inputCols = df.columns[1:], outputCol = 'vec_col')
df2_Vec = assem.transform(df)
indexer = StringIndexer(inputCol="Status", outputCol="statusIndex")
df_ml = indexer.fit(df2_Vec).transform(df2_Vec)

trainingData, testData = df_ml.randomSplit([0.7,0.3], seed=34) # Splits data
print("Training Data: ", trainingData.count())
trainingData.groupBy('Status').count().show()
print("Testing Data: ", testData.count())
testData.groupBy('Status').count().show()
```

```
Training Data:  697
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  346|
|  Normal|  351|
+--------+-----+

Testing Data:  299
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  152|
|  Normal|  147|
+--------+-----+
```

Figure 5 Split with Seed

Therefore, setting a seed as a parameter allowed me to generate such a split as shown in figure 5:

However, removing a seed as a parameter, continuously running this would generate different splits since it is random:

```
Training Data:  712
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  359|
|  Normal|  353|
+--------+-----+

Testing Data:  284
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  139|
|  Normal|  145|
+--------+-----+
```

```
Training Data:  693
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  350|
|  Normal|  343|
+--------+-----+

Testing Data:  303
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  148|
|  Normal|  155|
+--------+-----+
```

```
Training Data:  709
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  365|
|  Normal|  344|
+--------+-----+

Testing Data:  287
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  133|
|  Normal|  154|
+--------+-----+
```

```
Training Data:  699
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  356|
|  Normal|  343|
+--------+-----+

Testing Data:  297
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  142|
|  Normal|  155|
+--------+-----+
```

```
Training Data:  667
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  328|
|  Normal|  339|
+--------+-----+

Testing Data:  329
+--------+-----+
|  Status|count|
+--------+-----+
|Abnormal|  170|
|  Normal|  159|
+--------+-----+
```

Therefore, with the dataset consisting of 996 rows, this would be split to roughly 697.2 (70% training) and 298.8 rows (30% testing). Which as represented in figure 5, this would equate to [Abnormal: 346, Normal: 351] rows per status type for the training data, and [Abnormal: 152, Normal: 147] rows per status type for the testing data.

1.5 Classifiers

All the following calculations are based on the previous tasks random split under "*seed=34*", and are utilising the "*pyspark.ml*" library, with the following methods:

```
from pyspark.ml.classification import DecisionTreeClassifier, LinearSVC, MultilayerPerceptronClassifier
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
```

1.5.1 Decision Tree

Training

```
# Task 5

from pyspark.ml.classification import DecisionTreeClassifier, LinearSVC, MultilayerPerceptronClassifier
from pyspark.ml.evaluation import MulticlassClassificationEvaluator

# Decision Tree

decision_Tree = DecisionTreeClassifier(labelCol="statusIndex", featuresCol="vec_col")
model = decision_Tree.fit(trainingData)
```

Applying Trained Classifier to Test Set

```
x = model.transform(testData)
```

Calculating Sensitivity, Specificity, and Error Rate

```
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="accuracy")
tree_Accuracy = evaluation.evaluate(x.select("statusIndex", "prediction"))
print("Decision Tree:::\nAccuracy: ", tree_Accuracy)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="precisionByLabel")
tree_Sensitivity = evaluation.evaluate(x.select("statusIndex", "prediction"))
print("Sensitivity: ", tree_Sensitivity)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="recallByLabel")
tree_Specificity = evaluation.evaluate(x.select("statusIndex", "prediction"))
print("Specificity: ", tree_Specificity)
print("Error Rate: ", 1 - tree_Accuracy)
```

```
Decision Tree:::
Accuracy:  0.822742474916388
Sensitivity:  0.8113207547169812
Specificity:  0.8486842105263158
Error Rate:  0.17725752508361203
```

Decision Tree – Overall

```
# Decision Tree

decision_Tree = DecisionTreeClassifier(labelCol="statusIndex", featuresCol="vec_col")
model = decision_Tree.fit(trainingData)
x = model.transform(testData)

evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="accuracy")
tree_Accuracy = evaluation.evaluate(x.select("statusIndex", "prediction"))
print("Decision Tree:::\nAccuracy: ", tree_Accuracy)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="precisionByLabel")
tree_Sensitivity = evaluation.evaluate(x.select("statusIndex", "prediction"))
print("Sensitivity: ", tree_Sensitivity)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="recallByLabel")
tree_Specificity = evaluation.evaluate(x.select("statusIndex", "prediction"))
print("Specificity: ", tree_Specificity)
print("Error Rate: ", 1 - tree_Accuracy)
```

1.5.2 Support Vector Machine Model

Training

```
# Support Vector Machine

support_Vector_Machine = LinearSVC(maxIter=10, regParam=0.1, labelCol="statusIndex", featuresCol="vec_col")
vm_Model = support_Vector_Machine.fit(trainingData)
```

Applying Trained Classifier to Test Set

```
y = vm_Model.transform(testData)
```

Calculating Sensitivity, Specificity, and Error Rate

```
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="accuracy")
VM_accuracy = evaluation.evaluate(y.select("statusIndex", "prediction"))
print("\nSupport Vector Machine:::\nAccuracy: ", VM_accuracy)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="precisionByLabel")
VM_Sensitivity = evaluation.evaluate(y.select("statusIndex", "prediction"))
print("Sensitivity: ", VM_Sensitivity)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="recallByLabel")
VM_Specificity = evaluation.evaluate(y.select("statusIndex", "prediction"))
print("Specificity: ", VM_Specificity)
print("Error Rate: ", 1 - VM_accuracy)
```

```
Support Vector Machine:::
Accuracy:  0.7525083612040134
Sensitivity:  0.7349397590361446
Specificity:  0.8026315789473685
Error Rate:  0.24749163879598657
```

Support Vector Machine Model – Overall

```
# Support Vector Machine

support_Vector_Machine = LinearSVC(maxIter=10, regParam=0.1, labelCol="statusIndex", featuresCol="vec_col")
vm_Model = support_Vector_Machine.fit(trainingData)
y = vm_Model.transform(testData)

evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="accuracy")
VM_accuracy = evaluation.evaluate(y.select("statusIndex", "prediction"))
print("\nSupport Vector Machine:::\nAccuracy: ", VM_accuracy)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="precisionByLabel")
VM_Sensitivity = evaluation.evaluate(y.select("statusIndex", "prediction"))
print("Sensitivity: ", VM_Sensitivity)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="recallByLabel")
VM_Specificity = evaluation.evaluate(y.select("statusIndex", "prediction"))
print("Specificity: ", VM_Specificity)
print("Error Rate: ", 1 - VM_accuracy)
```

1.5.3 Artificial Neural Network

Training

```
ANN = MultilayerPerceptronClassifier(maxIter=100, layers=[12,5,4,2], blockSize=128, seed=1, labelCol="statusIndex", featuresCol="
ANN_Model = ANN.fit(trainingData)
```

Applying Trained Classifier to Test Set

```
z = ANN_Model.transform(testData)
```

Calculating Sensitivity, Specificity, and Error Rate

```
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="accuracy")
ANN_accuracy = evaluation.evaluate(z.select("statusIndex", "prediction"))
print("\nArtificial Neural Network:::\nAccuracy: ", ANN_accuracy)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="precisionByLabel")
ANN_Sensitivity = evaluation.evaluate(z.select("statusIndex", "prediction"))
print("Sensitivity: ", ANN_Sensitivity)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="recallByLabel")
ANN_Specificity = evaluation.evaluate(z.select("statusIndex", "prediction"))
print("Specificity: ", ANN_Specificity)
print("Error Rate: ", 1 - ANN_accuracy)
```

```
Artificial Neural Network:::
Accuracy:  0.7424749163879598
Sensitivity:  0.7027027027027027
Specificity:  0.8552631578947368
Error Rate:  0.2575250836120402
```

Artificial Neural Network – Overall

```
# Artificial Neural Network

ANN = MultilayerPerceptronClassifier(maxIter=100, layers=[12,5,4,2], blockSize=128, seed=1, labelCol="statusIndex", featuresCol="
ANN_Model = ANN.fit(trainingData)
z = ANN_Model.transform(testData)

evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="accuracy")
ANN_accuracy = evaluation.evaluate(z.select("statusIndex", "prediction"))
print("\nArtificial Neural Network:::\nAccuracy: ", ANN_accuracy)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="precisionByLabel")
ANN_Sensitivity = evaluation.evaluate(z.select("statusIndex", "prediction"))
print("Sensitivity: ", ANN_Sensitivity)
evaluation = MulticlassClassificationEvaluator(labelCol="statusIndex", predictionCol="prediction", metricName="recallByLabel")
ANN_Specificity = evaluation.evaluate(z.select("statusIndex", "prediction"))
print("Specificity: ", ANN_Specificity)
print("Error Rate: ", 1 - ANN_accuracy)
```

Task 2 – MapReduce for Margie Travel Dataset

2.1 Determine the Number of Flights from Each Airport, including a List of any Airports not used

```
# reading a CSV file into a Pandas DataFrame
pass_cols = ['PassengerID', 'FlightID', 'Airport Code', 'destinationCode', 'departTime', 'totalFlightTimeInMins']
df_Passengers_pd = pd.read_csv("AComp_Passenger_data_no_error.csv", encoding='latin1', names=pass_cols)

airport_cols = ['Airport Name', 'Airport Code', 'Latitude', 'Longitude']
df_Airports_pd = pd.read_csv("Top30_airports_LatLong.csv", encoding='latin1', names=airport_cols)

#df_Passengers_pd # displays the contents of the DataFrame in a row and column table format
#df_Airports_pd # displays the contents of the DataFrame in a row and column table format
```

```
# Task 1

# map
temp_df = df_Passengers_pd.drop_duplicates(subset='FlightID')
departure_Airport_Codes = temp_df['Airport Code'].tolist()
all_Airport_Codes = df_Airports_pd['Airport Code'].tolist()

# extracting only the departure airport codes for the flights
rdd = spark.sparkContext.parallelize(departure_Airport_Codes)
rdd2=rdd.map(lambda x: (x,1))
# mapping a '1' with every occurrence of an airport code into a list

# reduce
rdd_Reduce_By_Key = rdd2.reduceByKey(lambda a,b: a+b)
# reducing all of the occurrences of the same airport code into a summation of occurrences
print("\nNumber of Flights from each Airport:")
print("Airport Code | No. of Flights")

for element in rdd_Reduce_By_Key.collect():
    print(element[0], " | ", element[1]) # printing how many occurrences of each airport code

print("\nList of Airports not used: ")
missing_Airports = set(all_Airport_Codes).difference(departure_Airport_Codes)
# creates a list of all the airport codes that appear in the "Top30_airports_LatLong.csv" file, but do
#       a departure airport code in the "AComp_Passenger_data_no_error.csv" file.
print(missing_Airports)
```

```
Number of Flights from each Airport:
Airport Code | No. of Flights
DFW  |  1
PEK  |  1
JFK  |  1
PVG  |  1
FCO  |  1
BKK  |  1
DEN  |  3
ORD  |  2
MAD  |  1
AMS  |  1
MIA  |  1
KUL  |  2
CDG  |  1
HND  |  1
CGK  |  2
LAS  |  1
MUC  |  1
CAN  |  2
IAH  |  2
LHR  |  1
CLT  |  1
ATL  |  2

List of Airports not used:
{'IST', 'HKG', 'SIN', 'SFO', 'LAX', 'PHX', 'DXB', 'FRA'}
```

Figure 6 Task Output

Both initial "*AComp_Passenger_data_no_error.csv*" and "*top30_airports_LatLong.csv*" files were read into pandas DataFrames' with new column labels respectively.

The map part of the mapReduce was implemented in extracting only the airport departure codes from the passenger dataset, to indicate all of the departure flight's locations. All of these stored airport codes where then assigned an integer "1" under the "map" lambda and stored under a RDD DataFrame. This was intentional for the reduce section, where the RDD DataFrame's "*reduceByKey*" function would calculate a summation of all the entries of departure flights, storing them as a library of the key being the departure flight airport code and the value being the total sum of it's occurrences/records in the "*AComp_Passenger_data_no_error.csv*" file. To output this data, the reduced RDD DataFrame was then iterated through, printing out the results of "{airport code} | {number of flights from the airport code}" as shown in Figure 6.

2.2 Create a List of Flights based on the Flight ID

```
# Task 2

from functools import reduce
from datetime import datetime

# map
all_FlightIDs = df_Passengers_pd['FlightID'].tolist()
flights = df_Passengers_pd.FlightID.unique()
new_flights_df = pd.DataFrame(flights, columns=["FlightID"])
temp_flights_df = df_Passengers_pd[['FlightID','Airport Code', 'destinationCode','departTime', 'totalFlightTimeInMins']]
temp = temp_flights_df.groupby('Airport Code')['FlightID'].nunique()

# extracting only the FlightID's to represent how many passengers are on the same flight
rdd = spark.sparkContext.parallelize(all_FlightIDs)
rdd2=rdd.map(lambda x: (x,1))
# mapping a '1' with every occurrence of a FlightID into a list

# reduce
rdd_Reduce_By_Key = rdd2.reduceByKey(lambda a,b: a+b)
# reducing all of the occurrences of the same FlightID into a summation of occurrences

num_Passengers_df = pd.DataFrame(rdd_Reduce_By_Key.collect(), columns=['FlightID', 'NumOfPassengers'])
FlightInfoDF = temp_flights_df.drop_duplicates(subset=["FlightID"])
joint_pd = pd.merge(left=new_flights_df, right=FlightInfoDF, how='inner')
final_pd = pd.merge(left=joint_pd, right=num_Passengers_df, how='inner')
final_pd['departTime'] = pd.to_datetime(final_pd['departTime'], unit='s')
final_pd['Arrival Time'] = pd.to_datetime(final_pd['departTime']) + pd.to_timedelta(final_pd['totalFlightTimeInMins'], unit='m')
final_pd
```

	FlightID	Airport Code	destinationCode	departTime	totalFlightTimeInMins	NumOfPassengers	Arrival Time
0	SQU6245R	DEN	FRA	2015-01-06 17:14:20	1049	21	2015-01-07 10:43:20
1	XXQ4064B	JFK	FRA	2015-01-06 17:05:17	802	25	2015-01-07 06:27:17
2	SOH3431A	ORD	MIA	2015-01-06 17:00:49	250	18	2015-01-06 21:10:49
3	PME8178S	DEN	PEK	2015-01-06 17:13:29	1322	18	2015-01-07 15:15:29
4	MBA8071P	KUL	PEK	2015-01-06 17:04:16	572	16	2015-01-07 02:36:16
5	MOO1786A	MAD	FRA	2015-01-06 16:56:48	184	13	2015-01-06 20:00:48
6	HUR0974O	DEN	PVG	2015-01-06 17:15:25	1398	7	2015-01-07 16:33:25
7	GMO5938W	LHR	PEK	2015-01-06 17:11:57	1057	25	2015-01-07 10:48:57
8	DAU2617A	CGK	SFO	2015-01-06 17:23:06	1811	12	2015-01-07 23:34:06
9	RUM0422W	MUC	MAD	2015-01-06 16:58:59	194	14	2015-01-06 20:12:59
10	ATT7791R	AMS	DEN	2015-01-06 17:13:14	1001	15	2015-01-07 09:54:14
11	WPW9201U	DFW	PEK	2015-01-06 17:21:09	1452	11	2015-01-07 17:33:09
12	DKZ3042O	MIA	SFO	2015-01-06 17:05:27	538	11	2015-01-07 02:03:27
13	QHU1140O	CDG	LAS	2015-01-06 17:14:58	1133	21	2015-01-07 12:07:58
14	ULZ8130D	CAN	DFW	2015-01-06 17:23:03	1683	27	2015-01-07 21:26:03
15	VYU9214I	ORD	DXB	2015-01-06 17:18:36	1510	15	2015-01-07 18:28:36
16	HZT2506M	IAH	AMS	2015-01-06 17:12:04	1044	14	2015-01-07 10:36:04
17	EWH6301Y	CAN	DFW	2015-01-06 17:22:47	1683	10	2015-01-07 21:25:47
18	VYW5940P	LAS	SIN	2015-01-06 17:26:43	1843	17	2015-01-08 00:09:43
19	WSK1289Z	CLT	DEN	2015-01-06 16:59:02	278	21	2015-01-06 21:37:02
20	TMV7633W	CGK	DXB	2015-01-06 17:05:58	849	15	2015-01-07 07:14:58
21	FYL5866L	ATL	HKG	2015-01-06 17:25:40	1751	20	2015-01-07 22:36:40
22	BER7172M	KUL	LAS	2015-01-06 17:26:07	1848	17	2015-01-08 00:14:07
23	JVY9791G	PVG	FCO	2015-01-06 17:16:01	1189	20	2015-01-07 13:05:01
24	VDC9164W	FCO	LAS	2015-01-06 17:18:18	1276	15	2015-01-07 14:34:18
25	KJR6646J	IAH	BKK	2015-01-06 17:26:43	1928	23	2015-01-08 01:34:43
26	YZO4444S	BKK	MIA	2015-01-06 17:28:50	2027	17	2015-01-08 03:15:50
27	XIL3623J	PEK	LAX	2015-01-06 17:13:34	1302	13	2015-01-07 14:55:34
28	RPG3351U	HND	CAN	2015-01-06 16:59:29	374	13	2015-01-06 23:13:29
29	XOY7948U	ATL	LHR	2015-01-06 17:07:18	877	16	2015-01-07 07:44:18

This task involved utilising the lambda function to reduce the pandas dataframe to unique FlightID's, which would allow me to create a new dataframe under only a list of the flights occurring rather than individual passengers. Then to sum up all the records occurrences that matched with each of the unique FlightID's to get the number of passengers attending each flight.

2.3 Calculations

```
pip install geopy
```

```
Requirement already satisfied: geopy in c:\users\benwe\anaconda3\lib\site-packages (2.3.0)
Requirement already satisfied: geographiclib<3,>=1.52 in c:\users\benwe\anaconda3\lib\site-packages (from geopy) (2.0)
Note: you may need to restart the kernel to use updated packages.
```

```
# Task 3

from geopy.distance import great_circle

def combineFunction(departLat, departLong, destinLat, destinLong):
    depart = (departLat, departLong)
    destin = (destinLat, destinLong)
    return great_circle(depart, destin).miles

task_3_pd = final_pd.drop(["departTime","totalFlightTimeInMins", "NumOfPassengers", "Arrival Time"], axis=1)
task_3_pd = task_3_pd.merge(df_Airports_pd, left_on='Airport Code', right_on='Airport Code')
task_3_pd = task_3_pd.drop(["Airport Name"], axis=1)
task_3_pd = task_3_pd.rename(columns={"Latitude": "departLat", "Longitude":"departLong"})
task_3_pd = task_3_pd.merge(df_Airports_pd, left_on='destinationCode', right_on='Airport Code')
task_3_pd = task_3_pd.drop(["Airport Name", "Airport Code_y"], axis=1)
task_3_pd = task_3_pd.rename(columns={"Latitude": "destinLat", "Longitude":"destinLong"})
task_3_pd
temp_df = spark.createDataFrame(task_3_pd)
rdd2 = temp_df.rdd.map(lambda x: (x[0],x[1],x[2], combineFunction(x["departLat"], x["departLong"], x["destinLat"], x["destinLong'
df2 = rdd2.toDF(["FlightID", "Departure Airport Code", "Destination Airport Code", "Nautical Miles"])
task_3_pd = df2.toPandas()
print("\n\nNautical Miles for each flight: ")
display(task_3_pd)
```

```
Nautical Miles for each flight:
```

	FlightID	Departure Airport Code	Destination Airport Code	Nautical Miles
0	SQU6245R	DEN	FRA	5025.548293
1	XXQ4064B	JFK	FRA	3845.035111
2	MOO1786A	MAD	FRA	881.360918
3	PME8178S	DEN	PEK	6332.622466
4	MBA8071P	KUL	PEK	2742.402574
5	GMO5938W	LHR	PEK	5065.864311
6	WPW9201U	DFW	PEK	6956.134706
7	HUR0974O	DEN	PVG	6698.293743
8	SOH3431A	ORD	MIA	1199.271904
9	YZO4444S	BKK	MIA	9708.503899
10	VYU9214I	ORD	DXB	7233.101945
11	TMV7633W	CGK	DXB	4068.165581
12	BER7172M	KUL	LAS	8854.055622
13	QHU1140O	CDG	LAS	5428.871989
14	VDC9164W	FCO	LAS	6112.983233
15	DAU2617A	CGK	SFO	8675.125640
16	DKZ3042O	MIA	SFO	2580.989893
17	RUM0422W	MUC	MAD	929.359746
18	ATT7791R	AMS	DEN	4798.605881
19	WSK1289Z	CLT	DEN	1334.839982
20	ULZ8130D	CAN	DFW	8063.907681
21	EWH6301Y	CAN	DFW	8063.907681
22	HZT2506M	IAH	AMS	5001.604671
23	KJR6646J	IAH	BKK	9235.263242
24	VYW5940P	LAS	SIN	8829.255313
25	FYL5866L	ATL	HKG	8387.172912
26	XOY7948U	ATL	LHR	4200.818999
27	JVY9791G	PVG	FCO	5697.469902
28	XIL3623J	PEK	LAX	6236.802312
29	RPG3351U	HND	CAN	1792.571365

Following a similar approach to task 2, except reducing a RDD dataframe with a separate “combineFunction” to calculate the nautical distance by passing as arguments, the departure airport latitude, the departure airport longitude, the destination airport latitude, and the destination airport longitude. Which the result is then stored under a new column of the dataframe called “Nautical Miles”.

References

1. Escobar, Carlos A., et al. "Learning with Missing Data." *2020 IEEE International Conference on Big Data (Big Data)*, 10 Dec. 2020, 10.1109/bigdata50022.2020.9377785. Accessed 19 Jan. 2023.
2. Inada, Isshin. "PySpark DataFrame | RandomSplit Method." *Www.skytowner.com*, 5 July 2022, www.skytowner.com/explore/pyspark_dataframe_randomsplit_method. Accessed 19 Jan. 2023.
3. Zach. "How to Read a Correlation Matrix." *Statology*, 27 Jan. 2020, www.statology.org/how-to-read-a-correlation-matrix/.